



---

# COMPUTING

**9569/02**

Paper 2 (Lab-based)

**11 August 2026**

**3 hours**

Additional Materials:

- Electronic version of `MEMBERS.txt` file
- Electronic version of `FACILITIES.txt` file
- Electronic version of `BOOKINGS.txt` file
- Electronic version of `COMBATANTS.txt` file
- Electronic version of `EQUIPMENT.txt` file
- Electronic version of `STYLES.css` file
- Electronic version of `CLIENT.py` file
- Electronic version of `GAME_LOGIC.py` file
- Insert Quick Reference Guide

---

## READ THESE INSTRUCTIONS FIRST

Answer **all** questions.

All tasks must be done in the computer laboratory. You are not allowed to bring in or take out any pieces of work or materials on paper or electronic media or in any other form.

Approved calculators are allowed.

Save each task as it is completed.

The use of built-in functions, where appropriate, is allowed for this paper unless stated otherwise.

Note that up to **6** marks out of 100 will be awarded for the use of common coding standards for programming style.

The number of marks is given in brackets [ ] at the end of each question or part question.

The total number of marks for this paper is 100.

**Instruction to candidates:**

Your program code and output for each of Task 1 to 4 should be saved in a single `.ipynb` file. For example, your program code and output for Task 1 should be saved as:

`TASK1_<your name>_<centre number>_<index number>.ipynb`

Make sure that each of your `.ipynb` files shows the required output in Jupyter Notebook.

**Task 1 (34 marks)**

Name your Jupyter Notebook as:

`TASK1_<your name>_<centre number>_<index number>.ipynb`

A prototype for a turn-based combat game needs to be created using Object-Oriented Programming and a linked-list priority queue.

The game has a player and a series of enemies. Each combatant has a cooldown that determines how frequently they can act. Combatants act in order of readiness — the one closest to acting goes first.

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

In [1]:

```
#Task 1.1
Program code
```

Output:

**Task 1.1**

The class `Combatant` contains four attributes:

- `name` — a string for the combatant's name
- `health` — an integer for the combatant's current health points
- `damage` — an integer for the amount of damage the combatant deals per attack
- `cooldown` — an integer representing the number of turns between the combatant's actions

The class `Combatant` has the following methods:

- A constructor that takes `name`, `health`, `damage`, and `cooldown` as parameters.
- `take_damage()` that takes an integer `points` as a parameter and reduces `health` by that amount.
- `is_dead()` that returns `True` if `health` is less than or equal to zero, and `False` otherwise.

Write program code to declare the class `Combatant` and its methods.

[3]

## Task 1.2

A priority queue is implemented as a linked list using two classes: `Slot` and `PriorityQueue`. A missing node is represented by `None`. Each node in the priority queue has its priority represented by an integer: smaller integers represent higher priority.

The class `Slot` represents a single node in the linked list and contains the following attributes:

- `combatant` — the `Combatant` object stored in this slot
- `turns_before_ready` — an integer initialised to the combatant's `cooldown` value, representing how many turns must pass before this combatant can act
- `next` — a reference to the next `Slot`, initialised to `None`

It also contains the following methods:

- `get_priority()` returns `turns_before_ready` as the node's priority value.
- `update_priority()` that decrements `turns_before_ready` by 1 if it is greater than 0. `turns_before_ready` must not end up with a value less than 0.

Write program code to declare the class `Slot` and its methods.

[3]

The class `PriorityQueue` manages the linked list and contains the following attributes:

- `head` — a reference to the first `Slot` in the list, initialised to `None`
- `count` — an integer tracking the number of combatants in the queue, initialised to 0

It also contains the following methods:

- `get_combatants()` traverses the linked list and returns a list of all `Combatant` objects currently in the queue, without removing them.
- `add()` takes a `Combatant` as a parameter and inserts it into the linked list in ascending order of priority. If other nodes in the linked list have the same priority, the combatant is added after all existing slots with the same priority.
- `remove()` takes a string `name` as a parameter and removes the `Slot` whose combatant has that name from the linked list.
- `pop_highest_priority()` removes and returns the highest-priority `Combatant`, only if its `turns_before_ready` value is 0. The method returns `None` if the queue is empty or the highest-priority combatant is not yet ready.
- `update_slots()` calls `update_priority()` on every `Slot` in the linked list.

Write program code to declare the class `PriorityQueue` and its methods.

[9]

### Task 1.3

Two classes inherit from `Combatant`: `Player` and `Enemy`.

`Player` represents the human-controlled combatant and contains:

- the method `get_opponent_choice()` that takes a list of `Combatant` objects as a parameter, displays each with a numbered index, prompts the user to choose an opponent by inputting a number, re-prompts until a valid integer index is entered, and returns the chosen `Combatant`.

`Enemy` represents a computer-controlled combatant and contains:

- the method `get_opponent_choice()` that takes a list of `Combatant` objects as a parameter and returns the first `Combatant` in the list that is an instance of `Player`.

Write program code to declare the `Player` and `Enemy` classes and their methods.

[5]

### Task 1.4

The text file `combatants.txt` stores data about all combatants in the game in comma-separated format. The first row is a header. Each subsequent row contains:

- `name` — the combatant's name; the value "player" identifies the human player
- `health` — an integer
- `damage` — an integer
- `cooldown` — an integer

Write program code to:

- read the data from `combatants.txt`
- create a `Player` object for the row where name is "player", and an `Enemy` object for all other rows
- store a global reference to the `Player` object in a variable called `player`
- add each combatant (`Player` or `Enemy`) to a global `PriorityQueue`

[5]

**Task 1.5**

When the game is played, combatants act in turn order determined by the priority queue.

On each turn:

1. If `pop_highest_priority()` returns `None`, no combatant is ready — skip to step 6.
2. Otherwise, the active combatant chooses an opponent from the queue.
3. The active combatant deals its `damage` value to the chosen opponent, and a message is printed, for example: `player dealt 8 damage to mosquito!`
4. If the opponent is dead, an appropriate message is printed (e.g. `mosquito dies!`) and the opponent is removed from the queue.
5. The active combatant is re-added to the queue at the end of the turn.
6. Call `update_slots()` on the queue to advance time forward.
7. Repeat from Step 1 until all ready combatants have acted.

The game continues until the player is dead or is the only combatant remaining. After the game ends, print `"You died!"` if the player is dead, otherwise print `"You won!"`.

Write program code for the game loop and run it. [8]

Test your program by providing inputs until the game ends. [1]

Save your Jupyter Notebook for Task 1.

## Task 2 (12 marks)

A grid-based computer game is played over the network on a  $5 \times 5$  board where a player navigates blindly through coordinate updates to collect items or trigger hidden effects.

|        |  |  |  |        |
|--------|--|--|--|--------|
| (0, 0) |  |  |  | (0, 4) |
|        |  |  |  |        |
|        |  |  |  |        |
|        |  |  |  |        |
| (4, 0) |  |  |  | (4, 4) |

Objects on the grid can be classified into four categories as follows:

| Object    | Symbol | Effect                                                                                                              |
|-----------|--------|---------------------------------------------------------------------------------------------------------------------|
| Energiser | E      | Adds 10 points to the player's score.                                                                               |
| Shield    | S      | Gives the player one shield.                                                                                        |
| Toxin     | T      | Deducts 5 points from the player's score unless a shield is present, in which the shield will be destroyed instead. |
| Trap      | X      | Sends the player back to the starting location.                                                                     |

The game engine provided for you in `GAME_LOGIC.py` contains the `Game` class with the following methods:

- A constructor with the parameter `nickname` – instantiates the game engine and creates a player with the given nickname. Accessible attributes include:
  - `game.player.status()` – returns a formatted string of the player's current status (location, score, turns and shield status)  
eg. At (0, 0) | Score: 0 | Turns: 0 | Shield: False
  - `game.player.turns_taken` – an integer representing the number of valid turns completed
  - `game.max_turns` – an integer constant set to 8, representing the maximum number of valid turns allowed.
- `generate_items()` – randomly populates the grid with game items (`Energiser(E)`, `Shield(S)`, `Toxin(T)`, `Trap(X)`).
- `process_move(direction)` – takes a string direction ("UP", "DOWN", "LEFT", or "RIGHT"), processes player movement, applies item effects if landed on, and returns `True` if the move was valid, or `False` if invalid (eg. out of bounds).

The client program is provided in `CLIENT.py`.

## Task 2.1

Write program code to implement a game server using socket programming on the host `127.0.0.1` at port `65000`. The provided client interacts with your server according to the rules below:

1. The server uses classes from `GAME_LOGIC.py`.
2. The game will commence from the location `(0, 0)`.
3. The player may move one step per turn using the commands: `UP`, `DOWN`, `LEFT` or `RIGHT`, which are non-case sensitive.
4. The player cannot move out of the grid.
5. Invalid moves are rejected and do not count as a turn.
6. When the player visits a location containing an object, a gain or loss of points occurs, depending on the type of object at the location.
7. The game ends after the player has taken 8 valid turns.
8. The final score is displayed when the game ends.

Messages are sent between the client and the server using line-terminated strings, which are strings ending with the non-printable character `"\n"`. The server implementation uses the following protocol with the client:

- send the message `"WELCOME"` to the client
- send the message `"ENTER NAME"` to the client
- receive the player's nickname from the client
- instantiate a `Game` object using the player's nickname
- call the `generate_items()` method of the `Game` object to generate the grid objects
- send the message indicating the player's current status to the client
- send the message `"YOUR TURN"` to the client when a move is required
- receive a direction to move from the client
- process the move using the `process_move(direction)` method of the `Game` object
- continue the game until the player has taken 8 valid turns
- send the final score to the client when the game ends.

Save your Python program as:

`TASK2_1_<your name>_<centre number>_<index number>.py`

[12]

### Task 3 (23 marks)

Name your Jupyter Notebook as:

TASK3\_<your name>\_<centre number>\_<index number>.ipynb

A community sports centre manages its facilities and member bookings through a database. The sports centre offers various facilities (eg. swimming pool, badminton court, gym) that members can book for specific time slots. Each booking is made by one member for one facility. A member may make multiple bookings, and a facility may be booked many times.

The database is to be set up with the following tables:

Member:

- `MemberID` – The unique identifier for the member
- `FullName` – The full name of the member
- `ContactNumber` – The contact phone number of the member
- `MembershipType` – The tier or category of membership (eg. Standard, Premium)

Facility:

- `FacilityID` – The unique identifier for the facility
- `FacilityName` – The name of the facility (eg. Swimming Pool, Gym)
- `PricePerHour` – The hourly rental rate for the facility

Booking:

- `BookingID` – The unique identifier for the booking
- `MemberID` – The unique identifier of the member making the booking
- `FacilityID` – The unique identifier of the facility being booked
- `BookingDate` – The date of the booking
- `DurationHours` – The duration of the booking in hours

The sports centre stores its data in the following text files: `members.txt`, `facilities.txt` and `bookings.txt`. Each text file stores comma-separated values, with one record per line. The fields appear in the same order as the attributes listed in the table descriptions above.

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

In [1]:

```
#Task 3.1
Program code
```

Output:

**Task 3.1**

Write program code that uses SQL to create a database called `sports_centre.db` containing three tables: `Member`, `Facility` and `Booking`.

Define the primary and foreign keys for each table.

[6]

**Task 3.2**

Write program code that uses SQL to insert the data from each of the **three** text files into the appropriate fields in each database table.

[4]

Run your program to test the data has been entered correctly.

[1]

**Task 3.3**

Write program code for the procedure `show_member_bookings(member_id)` to:

- take a `member_id` as a parameter
- use SQL to retrieve all bookings made by that member, displaying the `FullName`, `FacilityName`, `BookingDate`, and `DurationHours` for each booking
- output the results in the following format:
  - o Alice Tan | Swimming Pool | 2025-06-01 | 2 hour(s)
  - o Alice Tan | Gym | 2025-06-02 | 1 hour(s)
- output "No bookings found." if the member has no bookings.

Test the procedure two times with the following `member_ids` and show the output:

M001, M009

[6]

Save your Jupyter Notebook for Task 3.

**Task 3.4**

Write a SQL query that shows:

- member name
- the total cost of all bookings made by that member, where:
  - $\text{Cost per booking} = \text{Price per hour} \times \text{Duration hours}$
- for the member with MemberID M001.

Save your SQL code as:

TASK3\_4\_<your name>\_<centre number>\_<index number>.sql

[6]

**Task 4 (25 marks)**

Name your Jupyter Notebook as:

TASK4\_<your name>\_<centre number>\_<index number>.ipynb

A community learning centre is developing a web application prototype to track and manage its equipment inventory using a hash table.

When inventory items are registered, each record stores two items:

- *Key field* – A positive integer representing a unique Equipment ID
- *Data item* – A single-word descriptor (eg. camera, projector, laptop).

The underlying hash table is implemented as a fixed size array, initialised to store 70 records.

The hashing algorithm is:

$$\text{hash}(k) = k \bmod 70$$

where  $k$  represents the Equipment ID key field and 70 is the maximum capacity of the hash table.

Empty hash table slots are represented by `None`.

For each of the sub-tasks, add a comment statement at the beginning of the code, using the hash symbol '#', to indicate the sub-task the program code belongs to, for example:

```
In [1]: #Task 4.1  
Program code
```

*Output:*

**Task 4.1**

Write program code to create a global list named `hash_tbl` containing 70 empty slots. [1]

**Task 4.2**

Write the function `get_hash()` that accepts an Equipment ID key as a parameter and returns its corresponding hash index, using the hash algorithm above. [2]

**Task 4.3**

Collisions are managed using open addressing with linear probing. Under this approach, consecutive table indices are inspected until an unoccupied slot is located or every index has been checked.

Write the function `add_record()` to:

- accept an Equipment ID key and its corresponding string value as parameters
- compute the initial index location using the function `get_hash()`
- insert the record into the list if the slot is open, or apply linear probing to locate the next open position
- return `True` upon successful insertion, or `False` if no available slots remain. [6]

**Task 4.4**

Write program code for the function `find_record()` to:

- accept a target Equipment ID as a parameter
- compute the initial index location using the function `get_hash()`
- search through the table using linear probing to account for collisions
- return the corresponding string value if the key exists, or `False` if the key is not present in the table. [3]

**Task 4.5**

The text file `EQUIPMENT.txt` contains 80 existing records. Each line contains a key field and a string data item separated by a single space.

Write program code for the function `load_data()` to:

- read all records from `EQUIPMENT.txt`
- insert each record into `hash_tbl` using `add_record()`
- display an appropriate error message if any equipment item cannot be stored
- display the total count of successfully loaded records upon completion.

Call the function `load_data()` and output the contents of `hash_tbl`. [5]

Save your Jupyter Notebook for Task 4.

**Task 4.6**

Write a Python program and the necessary files to create a web application that interacts with the populated hash table.

The web application must operate on a single web page with:

- the following menu options:
  - **View Inventory:**
    - displays all active (non-empty) equipment records stored in the `hash_tbl`.
  - **Search Equipment:**
    - allows users to input an Equipment ID
    - displays the matching equipment descriptor if the record is found or "Record not found." otherwise.
- the CSS styles applied from the file `STYLES.css`

Save your Python program as:

`TASK4_6_<your name>_<centre number>_<index number>.py`

with any additional files and sub-folders in a folder named:

`TASK4_6_<your name>_<centre number>_<index number>`

[6]

**Task 4.7**

Run the web application.

Test the **View Inventory** menu option.

Save the web page output as:

`TASK_4_7_1_<your name>_<centre number>_<index number>.html`

Test the **Search Equipment** menu option by querying the Equipment ID 475.

Save the web page output as:

`TASK_4_7_2_<your name>_<centre number>_<index number>.html`

[2]

**-- END OF PAPER --**